CENTRO PAULA SOUZA
FATEC OURINHOS
CURSO DE TECNOLOGIA EM JOGOS DIGITAIS

Guilherme Cunha Prada

# ALGORITMO GENÉTICO APLICADO AO COMPORTAMENTO DE UMA POPULAÇÃO DE NPCS EM UM JOGO DIGITAL

OURINHOS (SP)
2019

GUILHERME CUNHA PRADA

# ALGORITMO GENÉTICO APLICADO AO COMPORTAMENTO DE UMA POPULAÇÃO DE NPCS EM UM JOGO DIGITAL

Trabalho de graduação apresentado à Faculdade de Tecnologia de Ourinhos para conclusão do Curso de Tecnologia em Jogos Digitais.

Orientador: Prof. Me. Rodrigo Moura Juvenil Ayres

OURINHOS (SP)
2019

# RESUMO

Este trabalho tem como objetivo verificar a aplicabilidade de um algoritmo genético em um jogo digital, com a finalidade adicionar variedade de comportamentos a NPCs e adaptar a dificuldade ao jogador. Para tanto será implementada uma versão simplificada do jogo digital Pacman, à qual será adicionado um algoritmo genético, atuando na população de NPCs( Fantasminhas ).

**Palavras chave:** Jogos digitais, algoritmo genético, programação evolutiva, computação bioinspirada.

# ABSTRACT

This research aims to verify the viability of using a genetic algorithm in a computer game to add behavior variety to NPCs and adapt the difficulty to the player. The approach used was to implement a simplified version of the videogame Pacman and augment it with a genetic algorithm, acting upon the population of NPCs (ghosts).



**Keywords:** Computer games, videogames, genetic algorithm, evolutionary programming, bio-inspired computing.

# Sumário

**1.INTRODUÇÃO** ..... 6
2.1 O que são algoritmos Genéticos ..... 8
2.2 Cromossomo e os genes ..... 11
2.2.1 Função Fitness ..... 12
2.2.2 Seleção de candidato para reprodução ..... 12
2.2.3 Crossover ..... 13
2.2.4 Mutação ..... 13
2.2.5 Genes e sua representação ..... 13
2.3 Análise do jogo Pacman ..... 14
**3. METODOLOGIA** ..... 17
3.1 Lua ..... 18
3.2 Framework Love2d ..... 18
3.3 O Grid ..... 18
3.4 Estrutura básica do jogo ..... 19
3.5 O funcionamento dos fantasminhas ..... 21
3.6 Adaptando o Algoritmo Genético ..... 22
3.7 Monitoramento do Algoritmo Genético ..... 23
3.8 Medo e sistema de ações ..... 25
**4.RESULTADOS E DISCUSSÃO** ..... 29
**5.CONSIDERAÇOÊS FINAIS** ..... 35
**6.REFERÊNCIAS** ..... 36

# 1.INTRODUÇÃO

Algoritmos Genéticos ou GAs, são uma classe de algoritmos que explora a aplicação da seleção natural a entidades abstratas. Neste trabalho será estudado como funcionam os algoritmos genéticos com o objetivo de aplicar esta técnica em um jogo, para evoluir o comportamento de uma população de NPCs.

O jogo criado é inspirado no Pacman, ou o popular come-come. Este foi escolhido por sua representatividade, simplicidade e personalidade dos seus NPCs, os fantasminhas. Ao jogo original foram feitas algumas alterações e simplificações, entre elas a adição do GA, aumento da quantidade de NPCs e das dimensões da área de jogo.

**PROBLEMA**

O comportamento de NPCs em jogos digitais é normalmente fixo, ou seja, não se adapta a habilidade do jogador e se torna repetitivo.

**JUSTIFICATIVA**

NPCs que evoluem tem comportamento variável e são adaptáveis a diferentes situações, o que no contexto de um jogo pode resultar em um grau de dificuldade progressivo, que seja adaptado ao jogador e também no aumento da variação de comportamentos exibidos.

**OBJETIVO GERAL**

Este trabalho tem como objetivo verificar a aplicabilidade de um algoritmo genético em um jogo digital, com a finalidade adicionar variedade de comportamentos a NPCs e adaptar a dificuldade ao jogador. Com este intuito será implementada uma versão simplificada do jogo digital Pacman, à qual será adicionado um algoritmo genético, atuando na população de NPCs( Fantasminhas ). O jogo será desenvolvido utilizando prototipagem rápida, utilizando a linguagem Lua.

**OBJETIVOS ESPECÍFICOS**

- Estudar o conceito de algoritmo genético.
- Desenvolver um jogo com a linguagem de programação Lua, e o framework LOVE.
- Analisar a implementação dos fantasminhas no jogo Pacman.

# 2.REVISÃO BIBLIOGRAFICA

O interesse em explorar a evolução natural, na forma de vida artificial, existe desde o início da computação. As primeiras pesquisas na área tinham interesse em simular a natureza não somente para estudá-la, mas também para entender o funcionamento de seus mecanismos e talvez encontrar aplicações na área da computação.

Nas décadas de 50 e 60, sistemas evolucionários foram desenvolvidos independentemente por diversos cientistas, eles buscavam utilizar o mecanismo da evolução como forma de otimização para problemas específicos de engenharia, evoluindo uma população de possíveis soluções usando operações inspiradas na natureza (MITCHELL, 1999, p.2, tradução nossa).

Os Algoritmos genéticos foram inventados, durante os anos 60 por John Holland, que posteriormente os desenvolveu em conjunto com seus alunos, na Universidade de Michigan. Eles buscavam estudar formalmente o fenômeno da adaptação como ocorre na natureza e procurar formas de explorá-lo em sistemas computacionais. Seu livro Adaptação em Sistemas Naturais e Artificiais, de 1975 criou a base teórica dos GAs, inspirado na evolução natural (MITCHELL, 1999, p.3, tradução nossa).

## 2.1 O que são algoritmos Genéticos

GAs são uma classe de algoritmos que atua sobre uma população de indivíduos simulando o mecanismo da evolução natural, eles fazem parte da área da computação evolutiva, mas também são uma técnica de otimização bioinspirada. Devido aos diferentes objetivos de diferentes áreas ainda não é possível achar uma definição que seja amplamente aceita.

> Acontece que não existe definição rigorosa de Algoritmo Genético que seja aceita por toda comunidade e que diferencie GAs de outros métodos computacionais evolutivos. Entretanto, pode ser dito que a maioria dos métodos chamados de "GAs" tem ao menos os seguintes elementos em comum: populações de cromossomos, seleção de acordo com fitness, crossover para produzir a nova geração e mutação randômica da nova geração. Inversão --O quarto elemento de GAs de Holland—é

> raramente utilizado nos dias de hoje, e suas vantagens, se alguma, não são muito bem estabelecidas (MITCHELL, 1999, p.7, tradução nossa).

Disso tira-se a seguinte definição:

- Um algoritmo genético é um processo iterativo que atua em uma população de cromossomos usando os operadores genéticos de seleção, crossover e mutação.

O cromossomo codifica as características e define cada indivíduo. O operador seleção escolhe quais indivíduos irão participar da reprodução. O crossover é responsável pela troca de material genético entre os indivíduos escolhidos e a mutação adiciona um componente aleatório ao sistema.

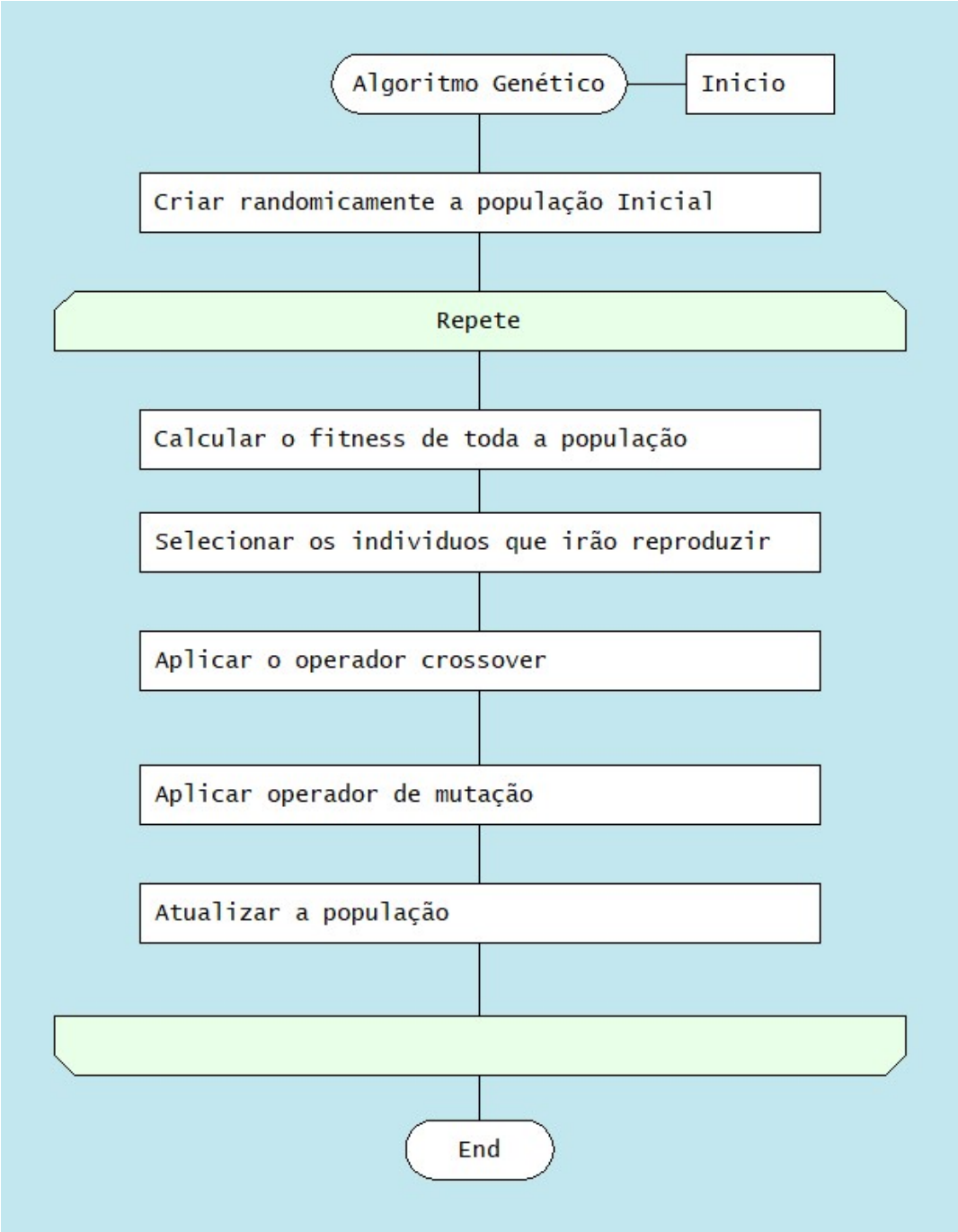
A seleção dos indivíduos que participarão da reprodução acontece de acordo com um valor atribuído a cada indivíduo, que é calculado por uma função chamada de Fitness, que é uma medida de desempenho do cromossomo e por isso é específica de cada implementação. Podemos entender melhor a participação da função Fitness em Haupt Randy L. e Haupt (2004, p.22) "O GA permite que uma população evolua sob regras especificas para maximizar o Fitness", ou seja, maximizar o Fitness médio da população é consequência da iteração do algoritmo genético. A escolha de uma boa função de Fitness é fundamental para o bom funcionamento do algoritmo pois é ela que define as características gerais que procuramos melhorar.

Estendendo à nossa definição:

- Um algoritmo genético é um processo iterativo que atua em uma população de cromossomos. Durante a iteração é associado a cada indivíduo um valor de Fitness, e com base neste número serão selecionados os participantes da etapa de reprodução, ou seja, a quais indivíduos serão aplicados os operadores de crossover e mutação para compor a próxima geração.

Com isso já é possível construir um fluxograma.

Imagem 1: Fluxograma de um Algoritimo Genético.




fonte: próprio autor.

## 2.2 Cromossomo e os genes

Os algoritmos genéticos funcionam processando uma população de cromossomos. Estes indivíduos podem representar coisas diferentes e de formas variadas de acordo com a necessidade. A cada indivíduo corresponde um cromossomo, que é um conjunto de genes que geralmente codificam diretamente algumas características do indivíduo, por exemplo uma cor, tamanho ou ação.

Para ilustrar, usa-se como uma população de cubos imaginária. Cada indivíduo no nosso sistema tem seis lados que podem ser de 10 cores diferentes (0-9). Para este exemplo essas são as únicas características que importam.

Uma possível forma para o cromossomo destes cubos seria uma sequência de seis números, com cada digito representando um gene (o zero tem significado em qualquer posição) e cada um destes genes representando a cor que será manifestada no lado do cubo correspondente. Um indivíduo poderia ser 123456, ou 378348, por exemplo.

No caso do controlador de um NPC é possível usar os genes correspondendo a ações de forma explicita, através de um identificador (CROCOMO, 2007). As ações propriamente ditas são implementadas usando métodos não evolutivos. Este modo limita as soluções a combinações de comportamentos pré-programados e por isso mesmo tem a vantagem de evitar que sejam desenvolvidos comportamentos indesejados. Pode-se também usar alguns dos genes para representar parâmetros que configurem o funcionamento das ações.

O algoritmo apresentado no fluxograma da imagem 1 possui um laço infinito, mas isso não será um problema na implementação do jogo deste trabalho, pois cada iteração do GA é acionada pelo laço principal somente quando há a solicitação de um novo indivíduo e, portanto, o algoritmo é interrompido junto com o jogo.

De forma geral, o algoritmo pode ser interrompido manualmente ou ser utilizado a convergência da população para alguma solução como critério de parada. No caso de um jogo, para adaptar um algoritmo genético, este será

estruturado de forma que o laço do GA seja controlado pelo laço do jogo (gameloop).

### 2.2.1 Função Fitness

Durante a iteração do GA é associado um valor de Fitness a cada cromossomo. Fitness pode ser traduzido do inglês como adequação, em um algoritmo genético será um número que de alguma forma sintetiza o desempenho do indivíduo a algum critério particular que será definido por ela mesma na sua forma de função, é, portanto, essencial que esteja em sintonia com o problema ao qual o algoritmo genético será aplicado.

No caso de um jogo, não há interesse em otimizar necessariamente, podemos pensar que estamos otimizando a diversão do jogador, mas isso é vago e difícil de expressar como uma função. Inimigos muito eficientes podem não se traduzir em mais diversão e isso de alguma forma tem que ser balanceado na função Fitness. Pode ser usado o tempo de vida do NPC ou o seu engajamento com o jogador ou uma combinação dos dois, por exemplo.

### 2.2.2 Seleção de candidato para reprodução

A seleção dos indivíduos para reprodução acontece com base no Fitness, quanto maior for o fitness tanto maior a chance do indivíduo ser selecionado (MITCHELL, 1999, p.8).

A forma como é composta a nova população vai além da seleção dos indivíduos, deverá também ser decidido o número de cruzamentos e quais indivíduos da geração anterior (se algum) serão preservados (elitismo). No caso de um sistema de jogo a população pode existir como um contínuo, sendo os indivíduos substituídos conforme outros saem do jogo. Esta é a forma usada no jogo criado, quando um fantasminha morre ele é substituído por um novo indivíduo gerado por seleção, crossover e mutação.

Existe ainda a possibilidade de não usar o valor do Fitness para a seleção dos indivíduos que irão reproduzir, desta forma a pressão seletiva é pelo tempo de vida, pois quanto maior este for, maior será a chance de o cromossomo ser eventualmente escolhido.

### 2.2.3 Crossover

O crossover é a etapa de reprodução em si. Nesta fase cada indivíduo participante fornecerá uma cópia do seu material genético ao qual será combinado outra cópia, que é fornecida por outro indivíduo. A escolha da maneira como é feita essa combinação é particular de cada implementação, deve ser feita de forma que as características gerais sejam preservadas e a combinação seja, em alguma medida, representativa dos cromossomos originais.

Uma forma simples seria o operador copiar os genes do pai até uma posição selecionada aleatoriamente, e copiar os genes que faltam da mãe (MITCHELL, 1999, p.8). Como ilustração, usa-se o mesmo sistema de cubos, enunciado anteriormente em 2.2 . Os genes do indivíduo pai são representados por P e os da mãe por M. Neste método tem-se duas formas possíveis de escolher quem será o pai e a mãe, o que não é um problema e pode ser aproveitado valendo-se das duas formas para criar mais um indivíduo. Usando a posição dois, e os indivíduos A=123000 e B = 573321, tem-se:

- A como pai e B como mãe,
  P = 12---- e M = --3321 que resulta em c=123321
- B como pai e A como mãe,
  P = 57---- e M=--3000 que resulta em c=573000

### 2.2.4 Mutação

A mutação é uma operação aplicada após o crossover ao indivíduo recém-criado, consiste em modificar alguns genes em posições escolhidas aleatoriamente (MITCHELL, 1999, p.8). É utilizada para que a população não fique presa em um máximo local, pois possibilita que genes extintos ou não presentes na população original apareçam.

### 2.2.5 Genes e sua representação

Os genes que serão usados pelo GA necessitam de uma forma de representação no programa, que é diferente do gene em si. No exemplo dos cubos, não foi sugerida uma forma de representação computacional e nada foi dito sobre o sistema ao qual estes estariam inseridos.

A forma usada para representação computacional do gene é escolhida de acordo com as necessidades da aplicação. Pode ser binária, uma estrutura com campos ou um vetor de números, por exemplo. No exemplo dos cubos, não sabemos nada sobre o sistema, mas pode-se dizer que a representação através de um vetor seria conveniente pois cada gene tem seu próprio índice e pode ser acessado diretamente, porem pode-se escolher de forma diferente.

A eficiência computacional da estrutura escolhida é relevante, já que algoritmos genéticos geralmente trabalham com um grande número de indivíduos e por sucessivas gerações. Nestas condições, pequenos ganhos são acumulados e por isso deve ser escolhida uma representação que propicie eficiência ao cálculo do Fitness e a operação de crossover. No contexto de um jogo digital o algoritmo deve ser executado sem causar interrupções ao fluxo do jogo, o que pode ser conseguido através de um algoritmo rápido o suficiente, ou escolhendo momentos nos quais o impacto ao jogo seja minimizado.

## 2.3 Análise do jogo Pacman

Para definir como seria o jogo foi usada uma análise do original. O Pacman Dossier (GAMASUTRA, 2019), é um compendio do jogo de autoria de Jamey Pittman. Este contém uma descrição do comportamento dos fantasminhas e da mecânica do jogo.

O jogo Pacman acontece em uma tela de jogo de 224x288 pixels, dividida em 28x36 células (GAMASUTRA, 2019, p.5) o que resulta em células de 8x8 pixels. A tela de jogo é um labirinto com corredores com uma célula de largura. O controlador de movimentação do Pacman e dos fantasminhas dependem do sistema de grid.

O personagem ganha pontos comendo as bolinhas na tela e/ou os fantasminhas e frutinhas. Só é possível pegar os fantasminhas por um curto período iniciado após o jogador pegar uma das pílulas disponíveis na fase. O resto do tempo os fantasminhas matam o personagem caso haja uma colisão.

O controle do personagem é feito por 4 comandos de direção e a movimentação ocorre de forma semiautomática. Quando o jogador pressiona uma das teclas de direção o personagem segue nesta direção até que aconteça uma mudança de direção ou uma colisão com a parede (bloco proibido).

Uma característica de jogabilidade é que o personagem pode mudar de direção um pouco antes de que esteja de fato sobre a célula de intersecção (GAMASUTRA, 2019, p. 4). Isto aumenta a janela de tempo para que o comando ocorra e permite que o personagem se mova ligeiramente mais rápido que os fantasminhas, mesmo tendo a mesma velocidade.

Existem 4 fantasminhas diferentes, cada um com comportamento próprio. De forma geral eles podem estar em 3 estados diferentes, dispersar(scatter), perseguir(chase) e assustado(frightened). Os fantasminhas ficam alternando entre perseguir e dispersar em intervalos de tempo pré-determinados e fazem a transição para o estado assustado quando o personagem ingere uma pílula (GAMASUTRA, 2019, p.3).

No estado de perseguir e dispersar, os fantasminhas movem-se em direção a uma célula específica do grid, a sua célula alvo. Esta é variável e calculada de acordo com o estado destes. No estado de dispersar a célula alvo é escolhida aleatoriamente em um dos cantos e cada fantasminha tem o seu próprio canto. Já no estado de perseguir a célula alvo é calculada usando a posição do personagem jogador, um dos fantasmas usa a própria célula do jogador e existe outro que usa a célula quatro casas à frente. Esses dois comportamentos são a inspiração da primeira versão do algoritmo genético, implementado na segunda etapa. Os dois comportamentos restantes foram ignorados.

O estado assustado faz os fantasminhas vagarem pelo mapa (seleção pseudoaleatória de caminho feita a cada intersecção), assim o jogador terá que persegui-los.

A forma que o fantasminha usa para se mover em direção a sua célula alvo funciona da seguinte forma, quando este chegar a uma intersecção será calculado qual das células que ele pode atingir que o coloca mais próximo da sua célula alvo e então se move na direção desta (GAMASUTRA, 2019, p.6).

Durante a movimentação, os fantasminhas são proibidos de inverter a direção, ou seja, em uma intersecção eles só podem seguir em frente ou virar para um dos lados, porém quando ocorre a mudança de estado para freightned ou de scatter para chase, ou chase para scatter eles invertem a direção (GAMASUTRA, 2019, p.3).

# 3. METODOLOGIA

O jogo foi produzido usando o framework LOVE, em linguagem Lua. Foi construído através de melhorias sucessivas ao projeto inicial. A execução do projeto foi dividida em etapas. Na primeira etapa foi implementado um protótipo funcional de um jogo similar ao Pacman, porém simplificado.

O jogo conta apenas com uma fase, que é semelhante a primeira fase do jogo original. Nesta fase temos o personagem, os fantasminhas e as pílulas. Não serão considerados, o sistema de progressão de fases, as frutinhas, o sistema de pontos, a possibilidade de passar de um lado a outro da tela e outros detalhes.

O jogo não tem condição de vitória e também não tem final, ele continua até que o jogador decida parar. Cada fantasminha que for capturado é substituído por outro. Desta forma não se tem um jogo propriamente, mas tem-se um ambiente propício a evolução dos NPCs. Poder-se-ia criar um jogo a partir desse sistema de algumas formas, uma em particular que foi testada foi desenvolvida usando um timer para limitar a criação do novo indivíduo. O objetivo do jogador passou a ser levar os NPCs a extinção, o que ocorre quando restar somente um fantasminha. Esse jogo pode ser obtido na versão final do projeto com a alteração da opção de configuração ghost_respawn_time.

A movimentação do personagem é inspirada no Pacman e tem a mesma mecânica de controle, como descrita na análise. A pílula capturada é substituída por uma nova imediatamente após seu efeito passar. A área de jogo é duas vezes maior, mantendo as características do original. Foi usado um mapa similar ao do Pacman repetido duas vezes, lado a lado.

A segunda etapa consistiu em estender o jogo da primeira fase para que fosse implementado o algoritmo genético. A gameplay permaneceu a mesma, sendo a única diferença a forma de substituição dos fantasminhas que são pegos, que desta vez é feita pelo algoritmo genético. Na terceira etapa foi criado o medo e o sistema de ações.

As versões finais de cada etapa podem ser simuladas através de opções no código. Essa característica foi adicionada para que a realização dos testes finais fosse feita no mesmo corpo de código.

### 3.1 Lua

Lua é uma linguagem de programação desenvolvida na PUC-Rio, ela é multiparadigma, apresenta tipagem dinâmica e pode ser usada como uma linguagem embarcada (LABLUA PUC-RIO, 2019). Possui uma única estrutura de dados chamada de tabela que é bastante flexível e pode funcionar como lista, dicionário ou um vetor por exemplo.

Existe mais de uma implementação da linguagem, sendo a PUC-Lua, a de referência, ela atualmente encontra-se na versão 5,3. O Love2d é implementado com a LuaJIT, que é uma versão que utiliza a compilação just-in-time para melhorar o desempenho. Ela atualmente é compatível com Lua 5,1.

### 3.2 Framework Love2d

Para construir o jogo foi utilizado o framework Love2d, que possibilita a criação de jogos em linguagem Lua. Este funciona através de callbacks, que são funções implementadas pelo jogo e que são chamadas automaticamente pelo framework quando necessárias. O framework é de código aberto, gratuito e está disponível sob as licenças zlib/libpng, podendo ser utilizado em projetos comerciais e de código fechado (Love2d WIKI, 2019).

### 3.3 O Grid

O grid é representado pela matriz de inteiros grid.grid_types[ ][ ], que armazena o tipo de cada célula e o formato do grid. A partir desta são criados a matriz grid.grid_directions[ ][ ], que armazena um vetor com as direções permitidas em cada célula e o vetor grid.grid_valid_pos[ ], que contém os índices do mapa que são permitidos.

A matriz grid.grid_types[ ][ ] também participa da renderização do maze_canvas, que é a imagem estática do mapa. Ela é gerada por código no carregamento do jogo, pela função love.load().

Existem 17(0-16) configurações possíveis para cada célula do grid, elas estão listadas na tabela 1. Duas delas, os números 0 ou 16 não permitem

movimento algum (blocos proibidos). Essa duplicidade foi feita para possibilitar a criação de padrões gráficos no maze_canvas, usando os blocos proibidos.

A representação adotada usou codificação binária para ser criada. Deixando de lado o número 16 (que neste caso pode ser mapeado para o 0), pode-se usar 4 bits, cada um representando uma direção (acima, abaixo, esquerda e direita). O zero representa que a direção é proibida e 1 que a direção é permitida, existem, portanto, os tipos de 0000 até 1111. Dessa forma pode-se recriar a codificação de forma consistente.

Tabela 1: Sistema de codificação do grid.

| binária | Decimal | Permitido |
|---|---|---|
| 0000 | 0 | Nenhuma |
| 0001 | 1 | Direita |
| 0010 | 2 | Esquerda |
| 0011 | 3 | direita e esquerda |
| 0100 | 4 | Abaixo |
| 0101 | 5 | abaixo e direita |
| 0110 | 6 | abaixo e esquerda |
| 0111 | 7 | abaixo, esquerda e direita |
| 1000 | 8 | Acima |
| 1001 | 9 | acima e direita |
| 1010 | 10 | acima e esquerda |
| 1011 | 11 | acima, esquerda e direita |
| 1100 | 12 | acima e abaixo |
| 1101 | 13 | acima, abaixo e direita |
| 1110 | 14 | acima, abaixo e esquerda |
| 1111 | 15 | acima, abaixo, esquerda e direita |
| 0000 | 16 | Nenhuma |

fonte: próprio autor.

### 3.4 Estrutura básica do jogo

A versão final do jogo ficou estruturada com a pasta raiz contendo scripts de inicialização, executáveis da Love2d e os arquivos de log. Os arquivos-fonte do jogo estão em uma subpasta chamada jogo, que contém os arquivos:

- Conf.lua, é um arquivo opcional de configuração da love2d.

- ghost.lua, contém o protótipo do fantasminha e funções para sua operação, o algoritmo genético está aqui, nas funções ghost.selection( ) e ghost.crossover( ).
- grid.lua, contém os dados e funções relativas ao Grid.
- main.lua, é o ponto de entrada do jogo, contém configurações, os callbacks da love2d e as funções para criar os arquivos de relatórios de desempenho do GA.
- pill.lua, contém o protótipo da pilula e funções para sua operação.
- player.lua, contém o protótipo do jogador e funções para sua operação.
- resizer.lua, contém funções para adaptar o jogo a diferentes resoluções.
- timer.lua, contém o protótipo do timer usado pelo jogo e funções para sua operação.
- utils.lua, contém diversas funções utilitárias.

Essa infraestrutura permite o desenvolvimento lado a lado das versões Windows e Linux e dispensa a necessidade de criar versões de lançamento.

No jogo foram utilizadas as seguintes callbacks:

- love.load( ), é carregada na inicialização do jogo, ela cria o jogador, os fantasmas, as pílulas e renderiza o maze_canvas a partir da matriz grid.grid_types[ ][ ].
- love.draw( ), é chamada quando é preciso desenhar a tela.
- love.update(dt), é chamada a cada ciclo do jogo, recebe o parâmetro dt, que representa o tempo desde a última atualização.
- love.keypressed(key, scancode, isrepeat), é chamada quando uma tecla é pressionada, é utilizada para controlar o menu do jogo.

Para armazenar os fantasmas e pílulas foram utilizados dois vetores, ghosts[ ] e pills[ ], os objetos contidos são iniciados por love.load( ) e só são destruídos com o fim do jogo. Quando algum deles sai de cena, ele é desabilitado, e posteriormente é redefinido e reabilitado. Usa-se a palavra objeto de forma ampla aqui, o jogo não é orientado a objetos, usou-se um sistema de protótipos, não há métodos ou herança, apenas funções e tabelas.

Essa configuração foi adotada para facilitar uma eventual migração do código para a linguagem C.

O objeto é criado por ghost.new( ) e quando é preciso redefini-lo usa-se ghost.regen( ), para criar um novo fantasminha aleatoriamente ou ghost.crossover( ), para usar o sistema genético.

### 3.5 O funcionamento dos fantasminhas

A movimentação dos fantasminhas ocorre de forma análoga ao original, conforme descrito na análise do jogo. A função love.update( ) chama ghost.update( ) e love.draw( ) chama ghost.draw( ), para cada um dos fantasminhas contido em ghosts[ ]. A lógica está contida na função ghost.update( ), que tem as seguintes etapas:

- Checar se o fantasma está ativo, se sim continua, senão retorna da função.
- Atualizar as propriedades do fantasminha.
- Resolver colisão com o jogador.
- Resolver colisão com as pílulas, determinar a mais próxima e fazer dela sua home.
- Resolver colisão com a parede.
- Verificar se houve mudança de célula do Grid.
- Verificar a distância para o centro do grid, caso seja pequena, centraliza o fantasminha e encontra o nova direção chamando a função ghost.find_next_dir( ), que implementa a lógica de movimentação.
- Determinar a velocidade a ser aplicada à movimentação.
- Mover o fantasminha de acordo com sua velocidade e direção.

A função ghost.find_next_dir( ) é que controla a direção adotada pelo o fantasminha. É ela que será explorada pelo algoritmo genético. Em linhas gerais funciona encontrando as direções que o fastaminha pode seguir e depois selecionando a que o coloque mais perto do personagem jogador.

No jogo original o alvo para o estado de scattering é um dos cantos da área de jogo, no início do projeto essa célula, chamada de home era fixa. Durante o desenvolvimento acabou sendo testada e adotada a opção de considerar dinamicamente a pílula mais próxima o home do fantasminha, o que tem o efeito de distribuí-los melhor no mapa.

A velocidade dos fantasminhas é variável, eles ganham um acréscimo de velocidade toda vez que colidem com alguma pílula, até um limite máximo. Isso foi feito para recompensar a interação do fantasminha com as pílulas.

### 3.6 Adaptando o Algoritmo Genético

O algoritmo genético está na função ghost.crossover( ) que é chamada quando for necessário criar um novo fantasminha, pelo laço principal do jogo. O Fitness é calculado pela função ghost.update( ) e é dinâmico.

A determinação dos genes foi feita a partir da análise da função ghost.find_next_dir( ), pois esta é a responsável pela escolha da próxima direção adotada pelo fantasminha. Os genes são parâmetros dessa função. Na análise procurou-se oportunidades para adicionar alguma configuração, que pudesse resultar em um comportamento diferente para cada fantasminha. O resultado foram genes:

- ghost.target_offset, indica a posição relativa ao player que o fantasma usa de alvo, números positivos estão afrente e negativos atrás.
- ghost.try_order, configura a ordem em que as direções são testadas.

No código existem alguns outros genes que ficaram por motivos históricos, eles são:

- ghost.pos_index, que inicialmente determinava a posição inicial e o home, perdeu seu efeito pois o home passou a ser determinado dinamicamente e a posição inicial passou a ser a mesma da mãe.
- ghost.pilgrim_gene, inicialmente esse gene condicionava a troca dinâmica de home, o que foi adotado como padrão para toda a população.

- ghost.target_offset_freightned, inicialmente indicava a posição relativa ao player que o fantasma usa de alvo no estado assustado, números positivos estão afrente e negativos atrás. Perdeu o efeito quando a ação de wander( ) foi definida como padrão para o estado freightned.

Existem ainda alguns outros genes que foram introduzidos na terceira etapa, para controlar o medo e as suas ações, estes serão discutidos posteriormente.

Efetivamente este algoritmo possui apenas dois genes, isso se deve ao fato deste ter sido concebido para minimizar mudanças no algoritmo do Pacman e facilitar a implementação. Em testes esse algoritmo acabou tendo uma performance inferior ao original.

A função de fitness utilizada pelo GA é bem simples, ela é a soma do valor do número de vezes que o fantasminha capturou o player e a taxa de colisão do fantasminha com pilulas. Da seguinte forma:

value.fitness = value.n_catches (value.n_pills*0.001)/value.n_updates.

O atributo value.n_catches é o número de vezes que o fantasminha capturou o player, value.n_pills é o número de ciclos que o fantasminha esteve em colisão com uma pílula e value.n_updates é quantos ciclos de update o fantasminha já passou.

A função de seleção é chamada pela função de crossover e pode operar de dois modos distintos. Quando a seleção por fitness estiver ligada a função seleciona o fantasminha pai aleatoriamente e a mãe também aleatoriamente, mas somente entre a metade de maior fitness da população. Com a seleção por fitness desligada a seleção do pai e da mãe são aleatórias e entre toda a população.

### 3.7 Monitoramento do Algoritmo Genético

O jogo conta com um sistema que muda algumas características da renderização do fantasminhas de acordo com os seus genes, esse sistema foi criado para dar diversidade ao jogo, mas também para ser usado como forma

de monitorar a população. Para essa finalidade esse sistema se mostrou útil no início do projeto, mas para os testes finais, acerto e verificação do funcionamento do jogo, foi criada uma infraestrutura para monitorar o desempenho do GA, ela consiste em arquivos de log que são criados na pasta raiz durante a execução do jogo. Estes arquivos são:

- config.run, este arquivo contém as configurações que foram usadas para o jogo. É criado na inicialização do jogo.
- fear_group.run, este arquivo contém uma linha para cada fantasminha capturado. Cada linha contém a distribuição do gene fear_group para aquele momento. É criado pela função reporter( ).
- fear_target.run, este arquivo contém uma linha para cada fantasminha capturado. Cada linha contém a distribuição do gene fear_target para aquele momento. É criado pela função reporter( ).
- gene_chase.run, este arquivo contém uma linha para cada fantasminha capturado. Cada linha contém a distribuição do chase_feared_gene. É criado pela função reporter( ).
- gene_scatter.run, este arquivo contém uma linha para cada fantasminha capturado. Cada linha contém a distribuição do scatter_feared_gene. É criado pela função reporter( ).
- stats.run, este arquivo contém uma linha para cada fantasma capturado. Cada linha contém o número de fantasmas capturados, o número vezes que o player foi capturado, a razão de fastasminhas capturados pelo número de vezes que o player foi capturado e por último a taxa de capturas por frame. É criado pela função reporter( ).
- target_offset_distribution, este arquivo contém uma linha para cada fantasminha capturado. Cada linha contém a distribuição do gene target_offset naquele momento. É criado pela função reporter( ).

Para monitorar a dificuldade do jogo serão usadas a razão de fantasminhas capturados pelo número de vezes que o player foi capturado (razão capturas/capturado) e a taxa de capturas por frame. Quanto mais baixa a razão capturas/capturado, que tem relação com a agressividade do jogador,

mais difícil é o jogo. O mesmo vale para a taxa de capturas por frame, quanto mais baixa, mais dificil, esta é relacionada com a dificuldade de capturar o fantasminha.

### 3.8 Medo e sistema de ações

O algoritmo genético como descrito até agora foi feito sem maiores alterações ao da etapa inicial, mas este se mostrou insuficiente para o uso proposto, com performance bastante inferior a versão randômica.

Para extender o algoritmo, os fantasminhas receberam o atributo medo e um aparato para controla-lo. Este sistema possibilita que o NPC tome decisões alternativas baseado em informações do meio. Para a implementação foram adicionados os genes fear_target, fear_group, chase_feared_gene e scatter_feared_gene, que controlam o gatilho do medo e as ações, respecivamente.

Tabela 2: Ações disponíveis e seu respectivo código no gene.

| Ações disponíveis | Ações Chase | Ações Scatter | Efeito |
|---|---|---|---|
| ghost.catch_target( ) | 7 | - | Usa o player como alvo. |
| ghost.go_to_target( ) | 6 | - | Usa o player como alvo, respeitando o target_offset. |
| ghost.surround_target_front( ) | 9 | - | Cerca o player pela frente. |
| ghost.surround_target_back( ) | 8 | - | Cerca o player por trás. |
| ghost.wander( ) | 5 | 5 | Movimento aleatório |
| ghost.go_home( ) | 1 | 1 | Vai para o home |
| ghost.go_to_group( ) | 3 | 3 | Vai para o centro do grupo |
| ghost.run_from_target( ) | 4 | 4 | Foge do jogador |
| ghost.go_to_closest_pill( ) | 2 | 2 | Vai para a pílula mais próxima, Equivalente a ghost.go_home( ). |

fonte: próprio autor.

O funcionamento do aparato ocorre da seguinte forma. O fantasminha terá o medo ativado se a distância do grupo for maior que seu gene fear_group e a distância do player for menor que o gene fear_target. Caso o medo esteja acionado cada fantasminha executará a ação correspondente ao

chase_feared_gene, se estiver em chase, ou scatter_feared_gene se estiver em scatter. Caso o medo esteja desativado o comportamento permanece o mesmo de anteriormente.

Os novos genes são representados por inteiros, os genes fear_target e fear_group tem valor entre 1 e 50 e são distancias medidas em unidades de grid. Os genes chase_feared_gene e scatter_feared_gene podem assumir os valores listados na tabela 2.

Para manter a comportamento próximo ao original foi respeitado o critério de executar somente manobras evasivas quando em scatter. A lista de ações para o scatter_feared_gene está contida na lista de ações para o chase_feared_gene. A ação no estado freightened permanece inalterada( ghost.wander( ) ).

Após a conclusão dessas alterações iniciou-se os testes finais, para calibrar e testar o comportamento do algoritmo.

### 3.9 Opções do Algoritmo Genético

Para acomodar as diferentes versões de gameplay a serem testadas, em uma única versão do código, o jogo possuí as seguintes opções localizadas em /jogo/main.lua:

- ghost_genetic_on, pode ser true ou false, liga e desliga a algoritmo genético, respectivamente.
- ghost_fitness_on, pode ser true ou false, liga e desliga a função fitness para a seleção de participantes na função crossover, respectivamente.
- ghost_target_offset_freightned_on, pode ser true ou false, liga e desliga o uso de um target_offset diferente como gene, para o estado freightned, respectivamente.
- ghost_migration_on, pode ser true ou false, liga e desliga. Quando ligado os fantasminhas migram a sua home para a pílula mais próxima, quando desligado eles usam a home fixa, determinada por pos_index.
- ghost_selective_migration_on, não foi usada na versão final.

- ghost_fear_on, pode ser true ou false, liga e desliga o sistema de medo dos fantasminhas, respectivamente.
- ghost_go_home_on_scatter, pode ser true ou false, liga e desliga. Quando ligado o comportamento no estado de scatter padrão é ghost.go_home( ), quando desligado é ghost.wander( ).
- ghost_chase_feared_gene_on, pode ser true ou false, liga e desliga o sistema de ações dos fantasminhas para o estado chase, respectivamente.
- ghost_scatter_feared_gene_on, pode ser true ou false, liga e desliga o sistema de ações dos fantasminhas para o estado scatter, respectivamente.
- ghost_target_spread, configura os limites para a criação randômica do gene target_offset, na população inicial. O sorteio é feito para valores entre –ghost_target_spread e ghost_target_spread.
- pill_genetic_on, pode ser true ou false, liga e desliga o sistema genético para as pílulas, respectivamente.
- pill_precise_crossover_on, pode ser true ou false, liga e desliga e sistema de cruzamento mais preciso para as pílulas genéticas.
- speed_boost_on, pode ser true ou false, liga e desliga. Quando ligado a velocidade de cada fantasminha sofre um acréscimo quando há colisão com uma pílula, até o limite dado por ghost_speed_max_factor. Quando desligado a velocidade dos fantasminhas permanece constante
- n_ghosts, configura o número de fantasmas.
- n_pills, configura o número de pílulas.
- pill_time, configura o tempo de duração do efeito das pílulas, em segundos.
- restart_pill_time, configura o tempo de duração do efeito da pílula de reinício, em segundos.

- ghost_chase_time, configura o tempo de duração do estado chase, em segundos.
- ghost_scatter_time, configura o tempo de duração do estado scatter, em segundos.
- ghost_respawn_time, timer utilizado para criação de um novo fantasminha.
- ghost_speed_max_factor, configura a velocidade maxima dos fantasminhas, é dada como fator da velocidade do personagem jogador.
- player_speed_grid_size_factor, configura a velocidade do jogador, é normalizada para o tamanho do grid.

# 4.RESULTADOS E DISCUSSÃO

O produto final, e os resultados dos testes finais podem ser verificados no repositório Git do jogo, disponível em https://github.com/guiprada/extincao-3. O jogo está localizado na subpasta /jogo e os resultados dos testes finais na subpasta /finais. Existem outros logs de testes, que foram usados durante a construção nas pastas /run-serie{1, 2, 3}. Os testes foram realizados pelo mesmo jogador, na mesma máquina e com a mesma versão do código, para minimizar variações.

Para estabelecer uma base de comparação foram feitos 4 testes com o algoritmo genético desligado, estes testes podem ser encontrados em /finais/sem_GA/. Em geral eles tiveram o mesmo resultado e os parâmetros medidos não tiveram variação. O primeiro teste (00) teve resultado um pouco diferente e a razão capturas/capturado piorou, o que pode ser atribuído ao fato de ser o primeiro teste e o jogador ter demorado um pouco para ser ambientar.

Tabela 3: Resultado de teste 01, sem o algoritmo genético.

| Fastasminhas criados | razão capturas/ capturado | taxa de capturas por frame |
|---|---|---|
| 20 | 6.6666666666667 | 0.0018026137899955 |
| 40 | 6.6666666666667 | 0.0018335166850018 |
| 60 | 5.4545454545455 | 0.0018698578908003 |
| 80 | 4.7058823529412 | 0.0017513901659442 |
| 100 | 5.2631578947368 | 0.0017958157493041 |
| 110 | 5.2380952380952 | 0.0017987082004742 |

fonte: próprio autor.

Tabela 4: Resultado de teste 02, sem o algoritmo genético.

| Fastasminhas criados | razão capturas/ capturado | taxa de capturas por frame |
|---|---|---|
| 20 | 3.3333333333333 | 0.0031811674884683 |
| 40 | 4.4444444444444 | 0.003427298432011 |
| 60 | 4 | 0.0032237266279819 |
| 80 | 3.6363636363636 | 0.0032149172158817 |
| 100 | 4 | 0.0030964545595293 |
| 120 | 4.8 | 0.0031598904571308 |
| 134 | 5.36 | 0.0032435310919081 |

fonte: próprio autor.

No teste final da versão simples do GA, resultado da segunda etapa, a população apresentou uma pequena melhora na taxa de capturas por frame, porém a razão de capturas/capturado piorou. Este resultado pode ser entendido como um aumento da dificuldade de capturar os fantasminhas e a diminuição da capacidade dos fantasminhas capturarem o personagem jogador e foi considerado insatisfatório, pois houve pouca melhora e bastante variação nos resultados. O resultado está sintetizado na tabela 5.

Tabela 5: Resultado de teste 00, do algoritmo genético inicial.

| Fastasminhas criados | razão capturas/ capturado | taxa de capturas por frame |
|---|---|---|
| 20 | 6.6666666666667 | 0.0058326042578011 |
| 40 | 8 | 0.0058351568198395 |
| 60 | 10 | 0.0054649785955005 |
| 80 | 11.428571428571 | 0.00549186517471 |
| 100 | 10 | 0.0047288031399253 |
| 120 | 7.0588235294118 | 0.0042347460916823 |
| 140 | 6.6666666666667 | 0.0041931232778244 |
| 160 | 7.2727272727273 | 0.0043117387086343 |
| 180 | 7.5 | 0.0044935966248096 |
| 200 | 8.3333333333333 | 0.0049001592551758 |
| 210 | 8.4 | 0.004747908659281 |

fonte: próprio autor.

Analisando a gameplay resultante deste algoritmo, entendeu-se que nenhuma das variações genéticas possíveis possuí diferenciação significativa de comportamento e que todas são capturadas de forma parecida. Valores de target_offset altos (menor precisão) mantém o fantasminha a uma distância maior do jogador o que confere alguma vantagem e estes sobrevivem por mais tempo. Este resultado foi um contrassenso e esperava-se que o aumento da precisão melhoraria o desempenho dos fantasminhas.

Seria preciso que os fantasminhas tivessem outra forma de defesa que não comprometesse a sua efetividade. Partindo desta conclusão foi criado o dispositivo medo, como extensão do algoritmo inicial. Este, que foi inicialmente formulado como sistema detector de emboscada, em consulta com o orientador, foi implementado como forma de possibilitar aos fantasmas assumirem comportamentos distintos em situações de "perigo”. Devido à forma

como foi feito, está relacionado a distância do jogador e distância do centro dos fantasmas, como descrito em medo e sistema de ações.

Na versão completa do GA foi possível observar o aumento da dificuldade do jogo pelos parâmetros taxa de capturados/frame e razão capturas/capturado, embora estes resultados não sejam melhores que os números obtidos nos testes com o sistema genético desligado, estes não são imediatamente comparáveis pois a taxa de atualização do jogo é variável. Nestes testes é possível notar o comportamento diferente das populações resultantes e a dificuldade se mostrou crescente, o que era o resultado desejado.

Em alguns testes foi observado que alguns indivíduos ganham a capacidade de esperar pelo grupo no estado de chase. Esse comportamento, que não foi diretamente planejado, acontece em indivíduos que tem valores intermediários nos genes fear_target e fear_group e possuem ações evassivas no gene chase_feared_gene.

Tabela 6: Resultado de teste 00-fear_and, do algoritmo genético final.

| Fastasminhas criados | razão capturas/ capturado | taxa de capturas por frame |
|---|---|---|
| 20 | 6.6666666666667 | 0.0071505184125849 |
| 40 | 13.333333333333 | 0.0077249903437621 |
| 60 | 12 | 0.0054729544832619 |
| 80 | 8.8888888888889 | 0.0060154898864576 |
| 100 | 10 | 0.0058132775258691 |
| 120 | 10 | 0.0056651874232839 |
| 140 | 9.3333333333333 | 0.0053418803418803 |
| 160 | 6.6666666666667 | 0.0046510275864074 |
| 180 | 6.2068965517241 | 0.0046149112911496 |
| 184 | 6.3448275862069 | 0.0046429472621751 |

fonte: próprio autor.

O primeiro resultado favorável pode ser verificado na pasta /finais/GA_full/00-fear_and e neste foi possível verificar aumento significativo da dificuldade na taxa de capturas por frame. A razão capturas/capturado piora inicialmente, porém começa a melhorar a partir do fantasminha 74. O resultado está sintetizado na tabela 6.

A partir dessa configuração foram feitos outros testes, variando o funcionamento do medo, o fitness e a velocidade do jogo, porém o resultado se mostrou bastante frágil e não foi verificado em outras configurações.

Para confirmar o resultado do teste /finais/GA_full/00-fear_and as mesmas configurações foram utilizadas em /finais/GA_full/07-rerun00, neste teste é possível verificar o aumento significativo e crescente da dificuldade na taxa de capturas por frame. A razão capturas/capturado só piorou. O resultado está sintetizado na tabela 7.

Tabela 7: Resultado de teste 07-rerun00, do algoritmo genético final.

| Fastasminhas criados | razão capturas/ capturado | taxa de capturas por frame |
|---|---|---|
| 20 | Inf | 0.0073921971252567 |
| 29 | 29 | 0.0062811349361057 |
| 70 | 6.3636363636364 | 0.0048372607283533 |
| 120 | 8 | 0.0043674479545785 |
| 170 | 9.4444444444444 | 0.0049339718473371 |
| 220 | 10.47619047619 | 0.0044158972300281 |
| 270 | 10.8 | 0.0043464962410857 |
| 320 | 11.034482758621 | 0.0040055075729128 |
| 370 | 10.277777777778 | 0.0039286889858673 |
| 420 | 10.769230769231 | 0.0038001827706952 |
| 470 | 11.190476190476 | 0.0037556434535938 |
| 520 | 11.818181818182 | 0.003769809624614 |
| 570 | 11.632653061224 | 0.0037585308760015 |
| 620 | 11.923076923077 | 0.0037065821725354 |
| 670 | 12.641509433962 | 0.003713866023669 |
| 720 | 13.090909090909 | 0.0037361063544942 |
| 770 | 13.508771929825 | 0.0036919654201888 |
| 820 | 13.898305084746 | 0.0037387609199176 |
| 870 | 13.809523809524 | 0.0037406483790524 |
| 887 | 14.079365079365 | 0.0037935975022988 |

fonte: próprio autor.

Nos testes 00 e 07,não foi usada a seleção por fitness, que foi ligada para o teste /finais/GA_full/08-rerun00-fitness_on. Este teve resultado misto pois, a população aumentou mínimamente de dificuldade, o que pode ser observado na taxa de capturas e no fantasminha 108 começou a perder dificuldade e o teste foi encerrado. Foi um resultado absoluto bom, porém com

mínima variação em parte porque o teste já iniciou com muita dificuldade. O resultado está sintetizado na tabela 8.

Tabela 8: Resultado de teste 08-rerun00-fitness_on, do algoritmo genético final.

| Fastasminhas criados | razão capturas/ capturado | taxa de capturas por frame |
|---|---|---|
| 20 | 10 | 0.0031735956839099 |
| 40 | 10 | 0.0039828736433337 |
| 60 | 7.5 | 0.0029344158067198 |
| 80 | 8 | 0.0029265437518291 |
| 100 | 10 | 0.0027590012415506 |
| 120 | 9.2307692307692 | 0.0026882924862225 |
| 140 | 8.75 | 0.0029292379796627 |
| 160 | 8.4210526315789 | 0.0028072637950697 |
| 180 | 9 | 0.0029166801698156 |
| 200 | 9.5238095238095 | 0.0028767458251226 |
| 220 | 9.5652173913043 | 0.0029720090781368 |
| 240 | 10 | 0.0030487030309189 |
| 260 | 10 | 0.0031237234783862 |
| 280 | 10.37037037037 | 0.0031998171533055 |
| 300 | 11.111111111111 | 0.0033505701553548 |
| 305 | 11.296296296296 | 0.003400715822806 |

fonte: próprio autor.

O teste /finais/GA_full/09-rerun00-fitness_on_target_spread_6 foi feito usando o target spread de 6, embora em absoluto o desempenho não tenha sido o melhor, este teste teve aumento da dificuldade nos dois parâmetros com crescimento quase constante. O resultado está sintetizado na tabela 9.

Com base nesses resultados acredita-se que é viável implementar um jogo com base no sistema desenvolvido, de forma que este exiba dificuldade progressiva e variações de comportamento. Para tal pode-se usar a mesma maneira que foi testada nas versões iniciais do programa, que consiste em usar um temporizador para limitar a criação dos fantasminhas. Há ainda outras maneiras, usando quantidade de vida ou tempo limitados e alguma forma de pontuação, como a quantidade de pílulas ou fantasminhas capturados, por exemplo.

Tabela 9: Resultado de teste 09-rerun00-fitness_on_target_spread, do algoritmo genético final.

| Fastasminhas criados | razão capturas/ capturado | taxa de capturas por frame |
|---|---|---|
| 20 | Inf | 0.0033534540576794 |
| 23 | 23 | 0.0035725380552967 |
| 50 | 50 | 0.0047732696897375 |
| 75 | 18.75 | 0.0043731778425656 |
| 100 | 16.666666666667 | 0.0040405672956483 |
| 125 | 20.833333333333 | 0.0042128677833575 |
| 150 | 25 | 0.0046513070172719 |
| 175 | 25 | 0.0043715027977618 |
| 200 | 18.181818181818 | 0.0041014703771302 |
| 225 | 16.071428571429 | 0.0041223891535361 |
| 250 | 17.857142857143 | 0.0040836995050556 |
| 275 | 17.1875 | 0.0040582618833287 |
| 300 | 15 | 0.0039747207758655 |
| 325 | 13.541666666667 | 0.0039013264509933 |
| 350 | 12.5 | 0.0038199598358509 |
| 375 | 12.931034482759 | 0.0038085758973005 |
| 400 | 12.903225806452 | 0.0038039085160002 |
| 425 | 12.878787878788 | 0.0037157495322527 |
| 450 | 12.162162162162 | 0.0036337796153039 |
| 475 | 11.585365853659 | 0.0036228844261732 |
| 500 | 12.19512195122 | 0.0037729015121789 |
| 525 | 11.666666666667 | 0.0036492291437866 |
| 550 | 11.95652173913 | 0.0037182009315783 |
| 575 | 12.234042553191 | 0.0037139424629575 |
| 600 | 12.244897959184 | 0.0037207687108157 |
| 625 | 11.574074074074 | 0.003794547993443 |
| 649 | 11.589285714286 | 0.0038166592176143 |

fonte: próprio autor.

# 5.CONSIDERAÇOÊS FINAIS

A parte de testes foi mais difícil e trabalhosa do que previsto. As sessões de jogo foram feitas de forma manual, pelo próprio desenvolvedor, intercaladas às tarefas de corrigir bugs e programar. Esta característica não é ótima pois quebra as iterações rápidas associadas ao fluxo de desenvolvimento e multiplica-se a isso as opções presentes na versão final do jogo. A existência de um jogador automático ou um sistema que pudesse salvar e carregar uma população teria minimizado o problema, assim como uma equipe de testes.

A linguagem Lua mostrou-se bastante apta para prototipagem rápida e não houve problemas relativos à performance. A linguagem ainda possui recursos para melhorar a estrutura do código que não foram explorados, tais como suporte a orientação a objeto e funções de primeira classe.

Para melhorar a legibilidade do código, poderia ser criado um sistema orientado a objetos com classes, ou usar ponteiros para função, por exemplo. Estes poderiam ser usados para criar um vetor de ações, adicionando a estas acesso indexado, isto simplificaria a função ghost.find_next_dir( ).

A criação e utilização de uma função map( ) reduziria o tamanho do código usada para as iterações sobre contêineres de objetos, operação frequente no projeto. Outro caminho que poderia ser explorado seria um sistema de atores e troca de mensagens, para diminuir o acoplamento dos objetos.

A refatoração seria recomendável para dar sequência ao projeto pois existem algumas opções no código que não são mais utilizadas e as possibilidades de melhoria da legibilidade do código citadas acima.

Algumas alterações que poderiam ser feitas são a criação de um sistema de ações por medo para o estado de freightned e a substituição do sistema genético baseado em target_offset por um sistema de ações. O jogo também poderia ser estendido para usar um sistema de câmera o que possibilitaria uma área de jogo maior e comportaria mais fantasminhas.

# 6.REFERÊNCIAS

**CROCOMO, Marcio Kassouf; MIAZAKI, Mauro; SIMÕES, Eduardo do Valle. Algoritmos evolutivos para a produção de NPCs com comportamentos adaptativos.** Anais.. São Leopoldo: UNISINOS, 2007. Disponível em: <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.657.4302&rep=rep1&type=pdf#page=52>.
Acesso em: 15 nov. 2019.

**GAMASUTRA. The Pac-Man Dossier.** 2019. Disponível em: <https://www.gamasutra.com/view/feature/132330/the_pacman_dossier.php>. Acesso em: 5 maio 2019.

**HAUPT RANDY L. E HAUPT, S. E. Practical Genetic Algorithms, second edition.** [S.l.]: John Wiley Sons, Inc, 2004.

**LABLUA PUC-RIO. Lua About.** 2019. Disponível em: <http://www.lua.org/about.html>. Acesso em: 5 maio 2019.

**LOVE WIKI. LOVE wiki.** 2019. Disponível em: <https://love2d.org/wiki/Main_Page>. Acesso em: 5 maio 2019.

**MITCHELL, M. An introduction to Genetic Algorithms.** [S.l.]: A Bradford Book, The MIT Press, 1999. Acesso em: 5 maio 2019.